04. 추적 생성과 패딩 (Trace Generation and Padding)

4.1 추적 생성 개요

4.1.1 추적이란?

추적(Trace)은 프로그램 실행의 완전한 기록입니다. 각 단계에서의 VM 상태를 행렬 형태로 저장합니다.

추적의 구조:

```
행(Row): 각 명령어 실행 단계

열(Column): 상태 변수들 (스택, 플래그, 추가 데이터)
```

4.1.2 왜 추적이 필요한가?

1. **검증 가능성**: 실행의 모든 단계를 기록하여 검증 가능
2. **증명 생성**: STARK 증명의 입력 데이터
3. **투명성**: 실행 과정을 완전히 재현 가능

4.2 VM 내부 추적 생성

4.2.1 VMState 기록

VM의 `run` 함수는 각 명령어 실행 후 상태를 기록합니다:

```rust
pub fn run(&mut self) -> Result<(), String> {
while self.ip < self.instructions.len() {
let instruction = self.instructions[self.ip].clone();
self.ip += 1;
// ... 명령어 실행 ...
// 상태 기록
self.trace.push(VMState {
stack: self.stack,
instruction: instruction,
extra_data: extra_data,
});
}
Ok(())
}
```

기록되는 정보:

- `stack` : 명령어 실행 후의 스택 상태
- `instruction` : 실행된 명령어
- `extra_data` : 추가 데이터 (나눗셈의 나머지 등)

4.2.2 내부 추적 예시

프로그램:

```rust
Push(10)
```

```
Push(20)
Add
```

내부 추적 (`Vec<VMState>`):

```
[
VMState {
stack: [10, 0, 0, 0],
instruction: Push(10),
extra_data: 0
},
VMState {
stack: [20, 10, 0, 0],
instruction: Push(20),
extra_data: 0
},
VMState {
stack: [30, 0, 0, 0],
instruction: Add,
extra_data: 0
}
]
```

4.3 추적 변환 (get_trace)

4.3.1 변환 과정 개요

`VM::get_trace` 함수는 내부 추적을 증명 시스템이 사용할 수 있는 형태로 변환합니다.

```
pub fn get_trace(&self) -> Vec<[Mersenne31; 11]> {
let mut final_trace: Vec<[Mersenne31; 11]> = vec![[Mersenne31::ZERO; 11]];
for i in self.trace.iter() {
match i.instruction {
Instructions::Push(val) => {
final_trace.push([
i.stack[0], i.stack[1], i.stack[2], i.stack[3],
val,
Mersenne31::ONE, // is_push
Mersenne31::ZERO, // is_add
Mersenne31::ZERO, // is_sub
Mersenne31::ZERO, // is_mul
Mersenne31::ZERO, // is_div
i.extra_data,
]);
}
// 다른 명령어들...
}
```

```
}

final_trace

}
```

4.3.2 초기 행 추가

```
let mut final_trace: Vec<[Mersenne31; 11]> = vec![[Mersenne31::ZERO; 11]];
```

왜 0으로 채워진 행을 추가하는가?

- AIR 제약 조건의 초기 상태 검증을 위해
- 첫 번째 행은 VM의 초기 상태를 나타냄
- 모든 값이 0 (빈 스택, 명령어 없음)

4.3.3 명령어별 인코딩

Push 명령어 인코딩

```
Instructions::Push(val) => {

final_trace.push([

i.stack[0], // 0: 스택 최상위

i.stack[1], // 1: 스택 두 번째

i.stack[2], // 2: 스택 세 번째

i.stack[3], // 3: 스택 네 번째

val, // 4: push_val (푸시할 값)

Mersenne31::ONE, // 5: is_push = 1
```

```
Mersenne31::ZERO, // 6: is_add = 0

Mersenne31::ZERO, // 7: is_sub = 0

Mersenne31::ZERO, // 8: is_mul = 0

Mersenne31::ZERO, // 9: is_div = 0

i.extra_data, // 10: extra_data

]);

}
```

Add 명령어 인코딩

```
Instructions::Add => {

final_trace.push([

i.stack[0], // 0: 결과 (a + b)

i.stack[1], // 1: 이동된 값

i.stack[2], // 2: 이동된 값

i.stack[3], // 3: 0

Mersenne31::ZERO, // 4: push_val = 0

Mersenne31::ZERO, // 5: is_push = 0

Mersenne31::ONE, // 6: is_add = 1

Mersenne31::ZERO, // 7: is_sub = 0

Mersenne31::ZERO, // 8: is_mul = 0

Mersenne31::ZERO, // 9: is_div = 0

i.extra_data, // 10: extra_data = 0

]);
```

```
}
```

Div 명령어 인코딩 (특별한 경우)

```
Instructions::Div => {
final_trace.push([
i.stack[0], // 0: 몫
i.stack[1], // 1: 이동된 값
i.stack[2], // 2: 이동된 값
i.stack[3], // 3: 0
Mersenne31::ZERO, // 4: push_val = 0
Mersenne31::ZERO, // 5: is_push = 0
Mersenne31::ZERO, // 6: is_add = 0
Mersenne31::ZERO, // 7: is_sub = 0
Mersenne31::ZERO, // 8: is_mul = 0
Mersenne31::ONE, // 9: is_div = 1
i.extra_data, // 10: 나머지 값!
]);
}
```

Div의 extra_data:

- 나눗셈의 나머지를 저장
- AIR 제약 조건에서 나눗셈 검증에 사용

- 예: 23 ÷ 10 = 2 나머지 3 → extra_data = 3

4.3.4 변환 예시

프로그램:

```
Push(10)
Add
```

변환 과정:

```
내부 추적:
VMState { stack: [10,0,0,0], instruction: Push(10), extra_data: 0 }
VMState { stack: [10,0,0,0], instruction: Add, extra_data: 0 }

↓ get_trace() 변환

최종 추적:
Row 0: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] // 초기 상태
Row 1: [10, 0, 0, 0, 10, 1, 0, 0, 0, 0, 0] // Push(10)
Row 2: [10, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0] // Add
```

4.4 추적 패딩 (Trace Padding)

4.4.1 왜 패딩이 필요한가?

STARK 증명 시스템은 추적의 행 수가 2의 거듭제곱이어야 합니다.

이유:

1. **FFT 효율성**: Fast Fourier Transform은 2의 거듭제곱 크기에서 최적
2. **다항식 보간**: 2의 거듭제곱 크기에서 효율적인 다항식 연산
3. **FRI 프로토콜**: FRI는 2의 거듭제곱 크기를 가정

4.4.2 패딩 알고리즘

`get_trace` 함수의 패딩 부분:

```
// 1. 필요한 2의 거듭제곱 계산
let mut index_pow = 1;
while (final_trace.len() / 11) > 2_usize.pow(index_pow) {
index_pow += 1;
}

// 2. 마지막 행의 스택 값 저장
let imi_trace = [
final_trace[final_trace.len() - 11], // 마지막 행의 stack[0]
final_trace[final_trace.len() - 10], // 마지막 행의 stack[1]
final_trace[final_trace.len() - 9], // 마지막 행의 stack[2]
final_trace[final_trace.len() - 8], // 마지막 행의 stack[3]
];

// 3. 패딩 행 추가
```

```
for _ in 0..(2_usize.pow(index_pow)) - (final_trace.len() / 11) {
final_trace.push(imi_trace[0]); // stack[0]
final_trace.push(imi_trace[1]); // stack[1]
final_trace.push(imi_trace[2]); // stack[2]
final_trace.push(imi_trace[3]); // stack[3]
for _ in 0..7 {
final_trace.push(F::ZERO); // 나머지는 0
}
}
```

4.4.3 패딩 과정 상세 분석

단계 1: 필요한 크기 계산

```
let mut index_pow = 1;
while (final_trace.len() / 11) > 2_usize.pow(index_pow) {
index_pow += 1;
}
```

예시:

```
현재 행 수: 5
5 > 2^1 (2)? Yes → index_pow = 2
5 > 2^2 (4)? Yes → index_pow = 3
```

```
5 > 2^3 (8)? No → 멈춤


결과: 2^3 = 8행으로 패딩
```

단계 2: 마지막 스택 상태 저장

```
let imi_trace = [
final_trace[final_trace.len() - 11],
final_trace[final_trace.len() - 10],
final_trace[final_trace.len() - 9],
final_trace[final_trace.len() - 8],
];
```

왜 마지막 스택 상태를 사용하는가?

- 프로그램 실행이 끝난 후의 최종 상태
- 패딩 행은 "아무것도 하지 않음"을 나타냄
- 스택은 유지되고 명령어 플래그는 모두 0

단계 3: 패딩 행 추가

```
for _ in 0..(2_usize.pow(index_pow)) - (final_trace.len() / 11) {
final_trace.push(imi_trace[0]); // stack[0] 유지
final_trace.push(imi_trace[1]); // stack[1] 유지
final_trace.push(imi_trace[2]); // stack[2] 유지
final_trace.push(imi_trace[3]); // stack[3] 유지
```

```
for _ in 0..7 {
final_trace.push(F::ZERO); // push_val, 플래그들, extra_data 모두 0
}
}
```

패딩 행의 구조:

```
[stack[0], stack[1], stack[2], stack[3], 0, 0, 0, 0, 0, 0, 0]
↑ 마지막 상태 유지 ↑ ↑ 모든 플래그 0 ↑
```

4.4.4 패딩 예시

원본 추적 (5행):

```
Row 0: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Row 1: [10, 0, 0, 0, 10, 1, 0, 0, 0, 0, 0]
Row 2: [20, 10, 0, 0, 20, 1, 0, 0, 0, 0, 0]
Row 3: [30, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
Row 4: [30, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] // 마지막 행
```

패딩 후 (8행):

```
Row 0: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Row 1: [10, 0, 0, 0, 10, 1, 0, 0, 0, 0, 0]
Row 2: [20, 10, 0, 0, 20, 1, 0, 0, 0, 0, 0]
```

```
Row 3: [30, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]

Row 4: [30, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] // 마지막 실제 행

Row 5: [30, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] // 패딩

Row 6: [30, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] // 패딩

Row 7: [30, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] // 패딩
```

4.4.5 패딩과 AIR 제약 조건

패딩 행도 AIR 제약 조건을 만족해야 합니다!

패딩 행 간의 전이:

```
Row 4 → Row 5:

- 모든 명령어 플래그가 0

- 스택 상태 동일

- 제약 조건: flag * (expression) = 0 * (anything) = 0 ✓
```

왜 만족하는가?

- 모든 명령어 플래그가 0이므로 모든 제약 조건이 자동으로 만족
- 스택 상태가 변하지 않으므로 일관성 유지

4.5 RowMajorMatrix 변환

4.5.1 get_trace 함수 (p3.rs)

```rust
pub fn get_trace<F: Field>(vm: VM) -> RowMajorMatrix<F> {
```

```rust
let trace = vm.get_trace();
println!("{:?}", trace);

let mut final_trace = Vec::with_capacity(trace.len() * 11);
for i in trace.iter() {
for j in i.iter() {
final_trace.push(F::from_canonical_u32(j.as_canonical_u32()));
}
}

// ... 패딩 로직 (위와 동일) ...

RowMajorMatrix::new(final_trace, 11)
}
```

4.5.2 필드 변환

```rust
final_trace.push(F::from_canonical_u32(j.as_canonical_u32()));
```

왜 필드 변환이 필요한가?

- VM은 `Mersenne31` 필드 사용
- 증명 시스템은 제네릭 필드 `F` 사용
- 호환성을 위해 변환 필요

4.5.3 RowMajorMatrix 생성

```
RowMajorMatrix::new(final_trace, 11)
```

RowMajorMatrix란?

- Plonky3의 행렬 표현
- 행 우선(Row-Major) 순서로 데이터 저장
- 효율적인 행 접근 가능

구조:

```
[row0_col0, row0_col1, ..., row0_col10,
row1_col0, row1_col1, ..., row1_col10,
...]
```

4.6 추적 생성 전체 흐름

```
1. VM 실행
↓
2. VMState 기록 (Vec<VMState>)
↓
3. VM::get_trace() 호출
↓
4. 초기 행 추가 (모두 0)
```

```
↓
5. 각 VMState를 11개 컬럼으로 변환
↓
6. 2의 거듭제곱 크기 계산
↓
7. 패딩 행 추가
↓
8. p3::get_trace() 호출
↓
9. 필드 변환
↓
10. RowMajorMatrix 생성
↓
11. STARK 증명 시스템에 전달
```

4.7 추적의 수학적 의미

4.7.1 추적을 다항식으로

STARK 증명 시스템은 추적을 다항식으로 변환합니다:

```
각 컬럼 → 하나의 다항식
행 인덱스 → 다항식의 평가 지점
```

예시:

```
컬럼 0 (stack[0]): [0, 10, 20, 30, 30, 30, 30, 30]

↓

다항식 p₀(x): p₀(0)=0, p₀(1)=10, p₀(2)=20, ...
```

4.7.2 패딩의 수학적 효과

패딩은 다항식의 차수를 제어합니다:

- 2의 거듭제곱 크기 → FFT 효율성
- 일정한 값으로 패딩 → 다항식의 안정성

4.8 추적 생성 최적화

4.8.1 메모리 효율성

```rust
let mut final_trace = Vec::with_capacity(trace.len() * 11);
```

- `with_capacity` 로 재할당 방지
- 메모리 사용량 예측 가능

4.8.2 패딩 최소화

- 필요한 최소 크기로 패딩
- 불필요한 행 추가 방지
- 증명 크기 최소화

4.9 요약

추적 생성과 패딩은 zkVM의 핵심 과정입니다:

1. **추적 생성**: VM 실행의 완전한 기록

2. **변환**: 증명 시스템이 사용할 수 있는 형태로 변환

3. **패딩**: 2의 거듭제곱 크기로 확장

4. **행렬화**: RowMajorMatrix로 최종 변환

이 과정을 통해 프로그램 실행이 수학적으로 검증 가능한 형태가 됩니다. 다음 문서에서는 이 추적을 사용하여 실제로 STARK 증명을 생성하는 과정을 살펴보겠습니다.